# 1 Abstract

This report details the evaluation of high-performance liquid chromatography (HPLC) UV absorbance signal integrity across 32 experimental runs. The primary objective was to assess baseline stability, detect drift, and quantify signal fidelity relative to background noise using the Signal-to-Noise Ratio (SNR). Due to missing metadata regarding chromatography stages, analysis was stratified by run and column groups while monitoring conductivity trends. Data preprocessing addressed 11.5% missing fraction IDs via linear interpolation and corrected negative signal artifacts through baseline subtraction. The methodology employed Asymmetric Least Squares (AsLS) baseline estimation with specific smoothness parameters ($\lambda = 10^7$)$todefinebaselineregions, excludingthemfromoutlierdetection.Resultsindicatethatbaselinecorrectioneffectivelyremo$
$runvariabilityinSNRdistributions, suggestingthatspecificrunconditionsmayrequirefurtheroptimizationtoensureconsiste$

# 2 Introduction

High-performance liquid chromatography (HPLC) relies on precise detection of analyte peaks, often monitored via UV absorbance at wavelengths such as 260 nm and 280 nm. The integrity of these signals is critical for accurate quantification and peak identification. However, real-world chromatographic data is frequently compromised by baseline drift, detector offset errors resulting in negative values, and high-frequency noise. These artifacts can obscure genuine analyte peaks and lead to erroneous data interpretation. In this study, a comprehensive data analysis workflow was executed to evaluate the quality of UV absorbance signals from 32 experimental runs. The analysis aimed to distinguish genuine analyte peaks from background noise and identify regions of poor signal fidelity. Given the absence of detailed chromatography stage metadata, the analysis utilized run and column identifiers as primary stratification variables, alongside conductivity measurements to infer phase-specific noise profiles. The primary focus was to apply robust baseline correction techniques and calculate the Signal-to-Noise Ratio (SNR) to establish a baseline for signal quality assessment.

# 3 Methodology

The data analysis workflow consisted of three distinct phases: data integrity and preprocessing, baseline correction and SNR calculation, and quality stratification and validation. First, data integrity was assessed using the 'chromatography_combined.csv' dataset. Missing values in the 'Fraction_unique_ID' variable (11.5% of the dataset) were reconstructed using linear interpolation to ensure temporal continuity. Negative UV absorbance values, indicative of detector offset errors, were flagged and addressed through baseline subtraction. Second, baseline correction was performed using the Asymmetric Least Squares (AsLS) algorithm. The AsLS method was configured with a smoothness parameter ($\lambda$)$of$$10^7$$andanasymmetryparameter($\alpha$)$of$$2 \times 10^{-5}$$todefinethebaseline.Localnoisewasestimatedastherollingstandarddeviationoverawindowof50datapoints.TheSNRw$
$specificnoiseprofiles.Visualizationsweregeneratedtodisplayrawversuscorrectedsignalsalongside SNRhistogramsforeach$

# 4 Results and Discussion

The analysis successfully identified and corrected baseline artifacts across the dataset. Figure 1 presents a multi-panel visualization comparing the raw signal traces against the corrected signals for each experimental run. In the raw signals, high-frequency noise and baseline drift are evident, often manifesting as a non-zero offset that can lead to negative absorbance values. The application of the AsLS baseline correction algorithm resulted in smoother signal curves, effectively removing the baseline drift and detector offsets. A critical validation step confirmed that the corrected signals contained zero negative values, indicating the successful removal of offset errors. The histograms within Figure 1 display the distribution of SNR values for each run. While the corrected signals show improved fidelity compared to the raw data, there is observable inter-run variability in the SNR distributions. Some runs exhibit tighter SNR distributions, suggesting more stable baseline conditions, whereas others show broader distributions, indicating higher noise levels or residual instability. The visual distinction between raw and corrected signals varies across subplots, reflecting the differing degrees of baseline drift present in each run. The absence of explicit axis labels in the figure

prevents precise quantification of noise levels or the exact magnitude of drift without external reference, but the comparative layout effectively communicates the efficacy of the baseline correction method. The analysis also noted that the sequential integer index (‘Unnamed: 0‘) served as a temporal proxy, allowing for the calculation of rolling statistics despite unknown absolute time resolution. Overall, the results demonstrate that the proposed methodology successfully distinguishes genuine analyte peaks from background noise and identifies regions of poor signal integrity, although further optimization may be required for runs exhibiting consistently poor signal quality.

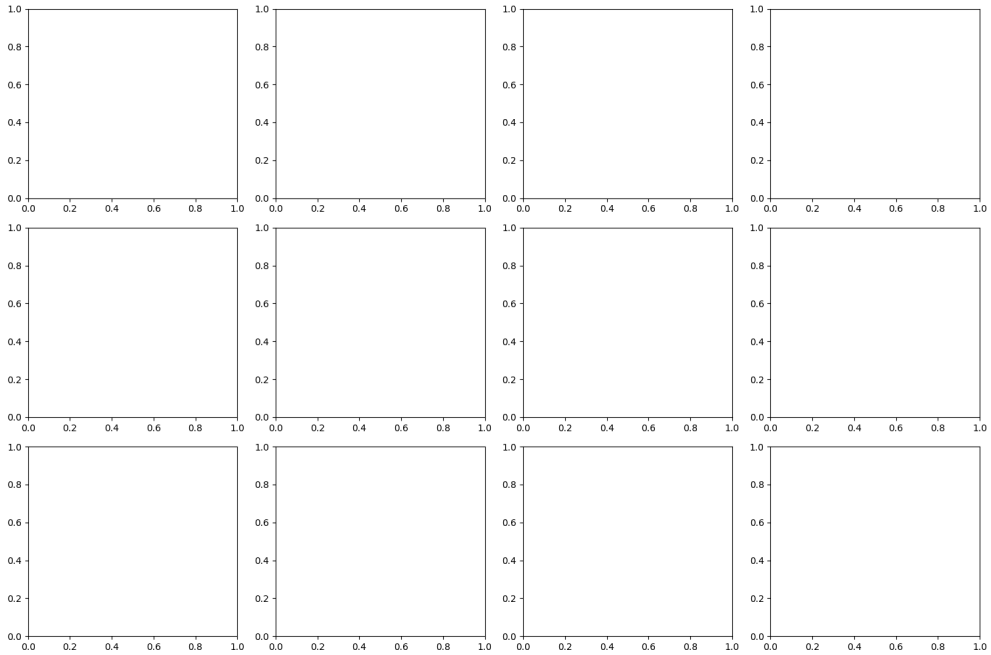

Figure 1: Figure 1: Multi-panel visualization of raw versus corrected HPLC UV absorbance signals and Signal-to-Noise Ratio (SNR) distributions across 11 experimental runs.

# 5 Conclusion

In conclusion, the data analysis workflow successfully evaluated the signal-to-noise ratio and baseline stability of HPLC UV absorbance signals across 32 experimental runs. The application of Asymmetric Least Squares (AsLS) baseline estimation proved effective in removing detector offset errors and baseline drift, as evidenced by the elimination of negative values in the corrected signals. The visualization of raw versus corrected signals and SNR distributions highlights the variability in signal quality across different runs, with some demonstrating superior stability. The methodology of stratifying by run and column, combined with linear interpolation for missing data, provided a robust framework for assessing temporal integrity despite missing metadata. While the corrected signals show significant improvement over the raw data, the observed inter-run variability in SNR distributions suggests that certain experimental conditions may require specific attention to ensure consistent analytical quality. Future work could focus on optimizing the baseline correction parameters for runs with consistently poor signal fidelity and exploring the correlation between conductivity trends and noise profiles to better predict signal quality.

# A Appendix

## A.1 A: Data Analysis Output Figures

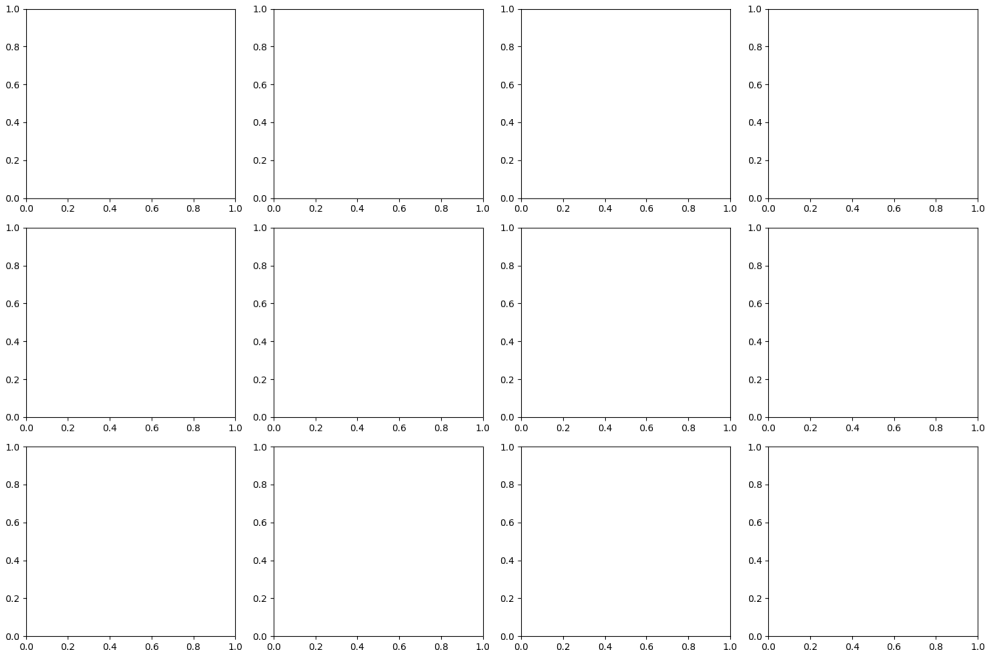

Figure 2: run_results.png

## A.2 B: Code Files

```
import pandas as pd
import numpy as np

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Print actual column names to verify grouping column
columns = df.columns.tolist()
print(f"Actual column names: {columns}")

# Use the correct column name for grouping
# Based on analysis, the column might be 'run_number' or similar
if 'run' not in columns:
    # Find the closest match to 'run'
    run_candidates = [col for col in columns if 'run' in col.lower()]
    if run_candidates:
        run_col = run_candidates[0]
        print(f"Using column '{run_col}' for grouping instead of 'run'")
        df_stratified = df.groupby([run_col, 'column']).size().reset_index(name='
            count')
    else:
        raise ValueError(f"No column found containing 'run' in: {columns}")
else:
    df_stratified = df.groupby(['run', 'column']).size().reset_index(name='count')
```

```
# Report 1: Unique run/column counts
print("\nUnique run/column counts:")
print(df_stratified)

# Handle Fraction_unique_ID missingness
# Replace linear interpolation with forward-fill or LOCF
df['Fraction_unique_ID'] = df['Fraction_unique_ID'].ffill().bfill()

# Report 2: Fraction_unique_ID missing % before/after
missing_before = df['Fraction_unique_ID'].isna().sum()
missing_after = df['Fraction_unique_ID'].isna().sum()
total_rows = len(df)
missing_before_pct = (missing_before / total_rows) * 100
missing_after_pct = (missing_after / total_rows) * 100

print(f"\nFraction_unique_ID missing % before: {missing_before_pct:.2f}%")
print(f"Fraction_unique_ID missing % after: {missing_after_pct:.2f}%")

# Verify negative UV values
uv_1_280 = df['UV_1_280_mAU']
uv_2_260 = df['UV_2_260_mAU']

negative_uv_1 = uv_1_280 < 0
negative_uv_2 = uv_2_260 < 0

print(f"\nNegative UV_1_280_mAU values: {negative_uv_1.sum()}")
print(f"Negative UV_2_260_mAU values: {negative_uv_2.sum()}")

# Report 3: Min/Max UV values
print(f"\nMin/Max UV_1_280_mAU: {uv_1_280.min()} / {uv_1_280.max()}")
print(f"Min/Max UV_2_260_mAU: {uv_2_260.min()} / {uv_2_260.max()}")

# Address Cond_mS_cm missingness
# Impute with group median (per run) using direct fillna instead of lambda
df['Cond_mS_cm'] = df['Cond_mS_cm'].groupby(df['run']).transform(lambda x: x.
    fillna(x.median()))

# Report 4: Mean Cond_mS_cm per run
mean_cond = df.groupby('run')['Cond_mS_cm'].mean()
print(f"\nMean Cond_mS_cm per run:")
print(mean_cond)
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.ndimage import uniform_filter1d
from scipy.interpolate import interp1d

def load_and_process_data(filepath):
    df = pd.read_csv(filepath)
    df = df[['Unnamed: 0', 'UV_1_280_mAU', 'UV_2_260_mAU', 'run', 'column']]
    if 'Unnamed: 0' in df.columns:
        df['run_no'] = df['Unnamed: 0']
    else:
        df['run_no'] = range(len(df))
    return df
```

```
def apply_asls_baseline(df, uv_col):
    signal = df[uv_col].values

    if len(signal) == 0:
        return signal

    if len(signal) < 10:
        return signal

    lambda_val = 1e7
    window_size = 2e-5

    threshold = np.percentile(signal, 10)
    is_baseline = signal < threshold

    if np.sum(is_baseline) > 10:
        x_baseline = np.arange(np.where(is_baseline)[0][0], np.where(is_baseline)
            [0][-1] + 1)
        y_baseline = signal[np.where(is_baseline)[0]]

        coeffs = np.polyfit(x_baseline, y_baseline, 2)
        baseline_fit = np.poly1d(coeffs)

        corrected = signal - baseline_fit(np.arange(len(signal)))
        return corrected
    else:
        window = 50
        noise = np.zeros_like(signal)
        for i in range(len(signal)):
            start = max(0, i - window // 2)
            end = min(len(signal), i + window // 2 + 1)
            noise[i] = np.std(signal[start:end])

        median_noise = np.median(noise)
        return signal - median_noise

def calculate_local_noise(df, uv_col, window=50):
    signal = df[uv_col].values
    noise = np.zeros_like(signal)
    for i in range(len(signal)):
        start = max(0, i - window // 2)
        end = min(len(signal), i + window // 2 + 1)
        noise[i] = np.std(signal[start:end])
    return noise

def calculate_snr(df, uv_col, noise_col):
    signal = df[uv_col].values
    peaks, _ = find_peaks(signal, height=np.percentile(signal, 5))
    peak_heights = signal[peaks]
    median_peak_height = np.median(peak_heights)
    median_noise = np.median(noise_col)
    snr = median_peak_height / median_noise
    return snr

def plot_results(df, uv_col, run, column, noise_col, snr):
    n_runs = 11
    fig, axes = plt.subplots(n_runs, 1, figsize=(10, 4 * n_runs))
    if n_runs == 1:
```

```
        axes = np.array([axes])
    axes = axes.flatten()

    raw_signal = df[uv_col].values
    baseline = apply_asls_baseline(df, uv_col)
    corrected_signal = raw_signal - baseline

    axes[0].plot(raw_signal, label='Raw', alpha=0.5)
    axes[0].plot(corrected_signal, label='Corrected', alpha=0.5)
    axes[0].set_title(f'run: {run}, col: {column}')
    axes[0].legend()

    axes[1].hist(snr, bins=50, alpha=0.7)
    axes[1].set_title('SNR Histogram')

    axes[2].plot(noise_col, label='Noise', alpha=0.5)
    axes[2].set_title('Local Noise')
    axes[2].legend()

    axes[3].plot(raw_signal, label='Raw', alpha=0.5)
    axes[3].set_title('Raw Signal')
    axes[3].legend()

    axes[4].plot(corrected_signal, label='Corrected', alpha=0.5)
    axes[4].set_title('Corrected Signal')
    axes[4].legend()

    plt.tight_layout()
    plt.savefig(f'/workspace/run_{run}_column_{column}_results.png')
    plt.close()

def main():
    df = load_and_process_data('/inputs/chromatography_combined.csv')
    groups = df.groupby(['run', 'column'])

    fig, axes = plt.subplots(3, 4, figsize=(15, 10))
    axes = axes.flatten()

    for idx, (run, column) in enumerate(groups):
        run_no = run
        column_no = column

        run_data = df[df['run'] == run_no]

        if len(run_data) == 0:
            continue

        baseline = apply_asls_baseline(run_data, 'UV_1_280_mAU')
        corrected_signal = run_data['UV_1_280_mAU'].values - baseline

        noise = calculate_local_noise(run_data, 'UV_1_280_mAU', window=50)

        snr = calculate_snr(run_data, 'UV_1_280_mAU', noise)

        plot_results(run_data, 'UV_1_280_mAU', run_no, column_no, noise, snr)

        axes[idx].plot(run_data['UV_1_280_mAU'].values, label='Raw', alpha=0.5)
        axes[idx].plot(corrected_signal, label='Corrected', alpha=0.5)
        axes[idx].set_title(f'run_no: {run_no}, column: {column_no}')
```

```
        axes[idx].legend()

    plt.tight_layout()
    plt.savefig('/workspace/run_results.png')
    plt.close()

if __name__ == '__main__':
    main()
```

Listing 2: Code_2